Artificial Intelligence

Lab 02 - Python Fundamentals

AI Course Team

Eng. Yousef Elbaroudy - Eng. Sahar Mohammed

Eng. Reham Ibrahim - Eng. Mawada Ashraf

Eng. Eman Nasser

2025/2026



What is Python ?

Python is a high-level, interpreted programming language known for its **simplicity and readability**. It was created by Guido van Rossum and first released in 1991. Python emphasizes code readability and allows developers to express concepts in fewer lines of code compared to languages like C++ or Java.

Key Features:

- (1) **Interpreted Language**: Python code is executed line by line, which makes debugging easier
- (2) **Versatile**: Python supports **multiple programming paradigms**, including procedural, object-oriented, and functional programming
- (3) **Dynamically Typed**: You don't need to **declare variable types explicitly**; Python infers them at runtime
- (4) **Extensive Libraries**: Python has a rich standard library and a vast ecosystem of third-party libraries (e.g., **NumPy**, Pandas, TensorFlow, Django) for various tasks

---

What is Jupyter Notebook ?



Jupyter Notebook is **an open-source web application** that allows you to create and share documents containing live code, equations, visualizations, and

narrative text. It is widely used for data analysis, machine learning, scientific computing, and educational purposes.

Key Features:

- (1) **Interactive Environment**: You can write and execute code in real-time, making it ideal for experimentation and exploration. In jupyter, you'll notice that the interface in **blocks-oriented**, each block called **CELL**.
- (2) **Support for Multiple Languages**: **While primarily used with Python (via the IPython kernel)**, Jupyter supports over 40 programming languages, including R, Julia, and Scala.
- (3) **Rich Output**: You can display outputs such as tables, charts, images, and videos directly in the notebook.
- (4) **Easy Sharing**: Notebooks can be exported to various formats (e.g., HTML, PDF, or slideshows) and shared with others.

---

(1) printing "Hello, World"

- In order to display output for anything you want, in which using python, you should use `print()` function
- **Note**: Python is case-sensetive language !

if you want to **RUN THE CELL** that you've just typed your code in, you have primarily **THREE OPTIONS**

1- `CTRL + ENTER`

2- `Shift + ENTER`

3- Using `run icon ->` from the ribbon 

The first one would only run the current cell you are selecting, while the second option will run the current cell **and make a new one below if not exist**

In [1]:
```
print("Hello World")
```

```
Hello World
```

In [2]:
```
# Take a look at the function name, it is mentioned before that PYTHON is case-sensetive language
Print("Hello World")
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[2], line 2
      1 # Take a look at the function name, it is mentioned before that PYTHON is case-sensetive language
----> 2 Print("Hello World")

NameError: name 'Print' is not defined
```

In [3]:
```
# you can display multiple outpus, just separate them using "comma ,"
print("Hello","World",'Yousef')
print("BFCAI")
```

```
Hello World Yousef
BFCAI
```

You may notice something strange, what is that blue line ?

```
# Take a look at the function name, it is mentioned before that PYTHON is case-sensetive language
```

- It is a **SINGLE-LINE COMMENT**, which is ignored by the interpreter during execution
- To make a comment line, just type `#` before the text

So, if there is a Single-line Comment, it should be there are Multi-line Comments !

- Not exactly, but you can do something similar using **MULTI-LINE STRING** but not ignored by the interpreter, just use **DOUBLED THREE DOUBLE QUTATION MARK** just like the following `""" """`

In [4]:
```
# This is single-line comment
"""
This is
Multi-line
String
and not
Ignored by
the interpreter
"""
```

Out[4]:
```
'\nThis is\nMulti-line\nString\nand not\nIgnored by\nthe interpreter\n'
```

(2) Variables declaration

- Unlike other programming languages, python has the capability to identify **DATA TYPES** due to the interpretability, just assign any value you want using **assignment operator** `=`

Variables names Rules

1. **Allowed Characters**

Variable names can contain:

```
Letters (a-z, A-Z)

Digits (0-9)

Underscores (_)
```

Example: `my_var`, `var1`, `user_name`

---

2. **Cannot Start with a Digit**

Variable names cannot begin with a number:

Valid: `name1`, `_var`

Invalid: `1name`, `2var`

---

3. **No Special Characters**

Variable names cannot contain spaces or special characters like !, @, #, $, %, etc.

Valid: `user_name`, `total_amount`

Invalid: `user-name`, `total$amount`

---

4. **Case-Sensitive**

Python is case-sensitive, so `myVar`, `myvar`, and `MYVAR` are treated as different variables.

---

5. **Cannot Use Python Keywords**

Variable names cannot be the same as Python's reserved keywords (e.g., if, else, for, while, class, etc.).

Example of invalid names: `if`, `for`, `class`

In [5]:
```
x = 5
y = 9
print(x + y)
```

```
14
```

In [6]:
```
3var = 23
```

```
  Cell In[6], line 1
    3var = 23
    ^
SyntaxError: invalid decimal literal
```

In [7]:
```
my-name = "Yousef Tarek"
```

```
  Cell In[7], line 1
    my-name = "Yousef Tarek"
    ^
SyntaxError: cannot assign to expression here. Maybe you meant '==' instead of '='?
```

In [8]:
```
# You can assign Multi variables in one line !
x, y = 2, 8.2
print(x + y)
```

```
10.2
```

(3) Input and Casting

- you may need to receive an input from user, you just could use **`input()`** function
-

Note: this function returns a **string** datatype of any input !

In [9]:
```
z = input("Enter a number: ")
print(z)
```

```
5
```

In [10]:
```
# type() function display the datatype of specific variable
type(x)
```

Out[10]: int

In [11]:
```
type(y)
```

Out[11]: float

In [12]:
```
type(z)
```

Out[12]:
```
str
```

How to use variable `z` in mathematical purposes ?

- In python, there is capability to cast any variable from to any datatype you want !

Basic Datatypes

1. **Int** - `int()`
2. **Str** - `str()`
3. **Float** - `float()`
4. **Boolean** - `bool()`

In [13]:
```
z = int(input("Enter a number: "))
print(z)
print(type(z))
```

```
14
<class 'int'>
```

In [14]:
```
myname = "yousef"
myname2 = 'elbaroudy'
print(myname,myname2)
print(type(myname),type(myname2))
```

```
yousef elbaroudy
<class 'str'> <class 'str'>
```

In [15]:
```
# Look at the following, you can't cast an actual string from str datatype to numerical datatype, since it is not base 10
myname = int(myname)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[15], line 2
      1 # Look at the following, you can't cast an actual string from str datatype to numerical datatype, since it is not base 10
----> 2 myname = int(myname)

ValueError: invalid literal for int() with base 10: 'yousef'
```

In [16]:
```
float_value = 4.5
print(type(float_value))
```

```
<class 'float'>
```

(4) Mathematical Operations and functions

Operators

- Addition (`+`)
- Difference (`-`)
- Multiplication (`*`)
- Division (`/`)
- Integer Division (`//`)
- Remainder/Modulus (`%`)
- Power (`**`)

Functions

- to use these, make sure to write this line `from math import *`

- `abs()`
- `ceil()`
- `min()`
- `max()`
- `floor()`
- `round()`
- `sqrt()`
- `sin()`
- `cos()`
- `log()`
- `log10()`

In [17]:
```
print(x + y)
print(13 - x)
print(120 * 32)
print(23 / 11)     # note the difference between / and // for the same operation
print(23 // 11)
```

```
print(10 % 3)
print(1.01**22)

from math import *
print(abs(-10))
print(ceil(12.43))
print(floor(92.51))
print(cos(30))
print(max(10,22))
```

```
10.2
11
3840
2.090909090909091
2
1
1.2447158597509211
10
13
92
0.15425144988758405
22
```

(5) Iterables and Iterators

In Python, an **iterable** is any object capable of returning its elements one at a time. It is a fundamental concept in Python that allows you to **LOOP** over (or iterate through) its elements.

an **iterator** is an object that represents a stream of data. It allows you to traverse through all the elements of a collection (or any iterable) one at a time. Iterators are used to implement iteration in Python, and they are the backbone of loops, comprehensions, and other iteration-based operations.

Common iterable categories in python:

- **Sequences**

1. Lists `[1, 2, 3]`
2. Tuples `(1, 2, 3)`
3. Strings `"Artificial Intelligence"`

- **Collections**

1. Sets `{1, 2, 3}`
2. Dictionaries `{"a":1 , "b":2}`

Common iterators in python:

1. `map()`
2. `enumerate()`
3. `zip()`
4. `reversed()`
5. `filter()`

In [18]:
```
lst1 = [1,2,3,4,5,6]
print(lst1)
print(type(lst1))
```

```
[1, 2, 3, 4, 5, 6]
<class 'list'>
```

In [19]:
```
# Heterogeneous: capability of storing different datatypes
lst2 = [1,3.25, 'Yousef',[10,11,12]]
print(lst2)
```

```
[1, 3.25, 'Yousef', [10, 11, 12]]
```

In [20]:
```
lst3 = list([9,4,2,1])
print(lst3)
```

```
[9, 4, 2, 1]
```

In [21]:
```
# accessing list using subscript
print(lst1[2])
lst1[1]
lst1[0]
```

```
3
```

Out[21]:
```
1
```

In [22]:
```
# get the length of the list
len(lst2)
```

Out[22]:
```
4
```

In [23]:
```
# Can be N-D (Any dimension)
# Matrices
matrix1 = [[1,2,3,4,5],[6,7,8,9,10]]
matrix2 = [[11,12,13,14,15],[16,17,18,19,20]]
```

```
print(matrix1,matrix2)
print(matrix1[0])
print(matrix1[1])
print(matrix1[0][1])
print(matrix1[1][3])
print("____________________")
print(len(matrix1))
print(len(matrix1[0]))
# 3D
three_d = [[[1,2,3],[4,5,6],[7,8,9]]]
print(three_d)

# Add matrices
add = [[None,None,None,None,None],[None,None,None,None,None]]
for i in range(len(add)):
    for j in range(len(add[0])):
        add[i][j] = matrix1[i][j] + matrix2[i][j]

print(add)
```

```
[[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]] [[11, 12, 13, 14, 15], [16, 17, 18, 19, 20]]
[1, 2, 3, 4, 5]
[6, 7, 8, 9, 10]
2
9
____________________
2
5
[[[1, 2, 3], [4, 5, 6], [7, 8, 9]]]
[[12, 14, 16, 18, 20], [22, 24, 26, 28, 30]]
```

In [24]:
```
# reverse access approach
lst2[-4]
```

Out[24]: 1

In [25]:
```
# mutable: the capability to modify the content of list
lst2[3] = 2
lst2
```

Out[25]: [1, 3.25, 'Yousef', 2]

In [26]:
```
tup1 = (1,2,3,4)
tup1
```

Out[26]: (1, 2, 3, 4)

In [27]:
```
tup1[1]
```

Out[27]: 2

In [28]:
```
# immutable: can't modify the content of tuples
tup1[1] = 9
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[28], line 2
      1 # immutable: can't modify the content of tuples
----> 2 tup1[1] = 9

TypeError: 'tuple' object does not support item assignment
```

In [29]:
```
# Tricky !!!
tup2 = (1,6.25 ,[1,2,3])
```

In [30]:
```
tup2[2][0] = 2
tup2
```

Out[30]: (1, 6.25, [2, 2, 3])

In [31]:
```
lst4 = [6,2,1,9,10]
lst4
```

Out[31]: [6, 2, 1, 9, 10]

In [32]:
```
list(reversed(lst4))
```

Out[32]: [10, 9, 1, 2, 6]

Note: the `reversed()` returns an iterator, so it has to be casted first to show the content of it

In [33]:
```
print(reversed(lst4))
```

```
<list_reverseiterator object at 0x000001E5C3586140>
```

In [34]:
```
# map() is appling a function to an iterable
x, y ,z = map(int, input("Enter three numbers: ").split())
print(x, y, z)
```

```
1 2 3
```

In [35]:
```
temp_list = [2,4,5,6]
```

```
squared = list(map(lambda x: x**2, temp_list))
print(squared)
```

```
[4, 16, 25, 36]
```

The same as `reversed()`, the `map()` should be casted first to show its content

In [36]:
```
map(lambda x: x**2, temp_list)
```

Out[36]:
```
<map at 0x1e5c3585ea0>
```

In [37]:
```
# string could be splitted into list, you should determine the delimiter to split (here is space which is default)
# list could be unpacked into variables
a, b, c = input("Enter three numbers: ").split()
print(type(a),type(b),type(c))
print(type(x),type(y),type(z))
```

```
<class 'str'> <class 'str'> <class 'str'>
<class 'int'> <class 'int'> <class 'int'>
```

Unpacking: a powerful feature that allows you to assign elements of an iterable (like a list, tuple, or dictionary) to individual variables in a single statement. It simplifies code and makes it more readable by eliminating the need for manual indexing or looping.

In [38]:
```
w, s, a = [10,20,30]
print(w,s,a)
```

```
10 20 30
```

In [39]:
```
"hello world".split()
```

Out[39]:
```
['hello', 'world']
```

You should try **List slicing**

List slicing: a technique in Python that allows you to extract a portion (or slice) of a list using a specific range of indices instead of accessing only one element.

`list[start (included): end (excluded) : step (default 1)]`

Note: **end** index should be greater than **start** index, unless the **step** parameter is decremental

In [40]:
```
# List slicing
print(lst3)
print(lst3[0:3:2])
print(lst3[0:3])
print(lst3[-1:0])
print(lst3[-1:0:-1])
```

```
[9, 4, 2, 1]
[9, 2]
[9, 4, 2]
[]
[1, 2, 4]
```

In [41]:
```
print(lst3)
lst3[-1:-3:-1]
```

```
[9, 4, 2, 1]
```

Out[41]:
```
[1, 2]
```

In [42]:
```
# if there is no start and no end, it will copy the entire list
lst1 = [1,2,3,4,5,6]
print(lst1)
lst9 = lst1[:]
print(lst9)
```

```
[1, 2, 3, 4, 5, 6]
[1, 2, 3, 4, 5, 6]
```

In [43]:
```
lst1[0] = 8
print(lst1)
print(lst9)
```

```
[8, 2, 3, 4, 5, 6]
[1, 2, 3, 4, 5, 6]
```

In [44]:
```
lst10 = lst1.copy()
```

In [45]:
```
lst1[0] = 15
print(lst1,lst10,lst9)
```

```
[15, 2, 3, 4, 5, 6] [8, 2, 3, 4, 5, 6] [1, 2, 3, 4, 5, 6]
```

In [46]:
```
# Concatenation
lst1 + lst10
```

Out[46]:
```
[15, 2, 3, 4, 5, 6, 8, 2, 3, 4, 5, 6]
```

In [47]:
```
# Repetition (not multiplication) !
lst1 * 3
```

Out[47]:
```
[15, 2, 3, 4, 5, 6, 15, 2, 3, 4, 5, 6, 15, 2, 3, 4, 5, 6]
```

In [48]:
```
["Hello !"] * 10
```

Out[48]:
```
['Hello !',
 'Hello !',
 'Hello !',
 'Hello !',
 'Hello !',
 'Hello !',
 'Hello !',
 'Hello !',
 'Hello !',
 'Hello !']
```

In [49]:
```
# Membership (can be used with if-else)
print(lst1)
print(2 in lst1)
print(100 in lst1)
```

```
[15, 2, 3, 4, 5, 6]
True
False
```

- There is some important **methods** to know about lists !

1. `.append()` - add an element to the end of list
2. `.insert()` - add an element to a specific index of list
3. `.pop()` - to remove an element of the end of list (default) or from a specific index
4. `remove()` - to remove the first occurance of an element from a list (the value itself if exists but not the index !)
5. `index()` - to lookup the index of an element in the list

In [50]:
```
lst1 = [1,1,2,2,3,3,4,4,5,5,6,6,7,7]
print(lst1)
lst1.append(23)
print(lst1)
lst1.insert(3, 2102)
print(lst1)
lst1.pop()           # remove the last element
print(lst1)
lst1.pop(3)          # remove the element at index 3
print(lst1)
lst1.remove(4)       # remove the first occurance of the value 4
print(lst1)
lst1.remove(122)     # raise an error that 122 is not in the list !
```

```
[1, 1, 2, 2, 3, 3, 4, 4, 5, 5, 6, 6, 7, 7]
[1, 1, 2, 2, 3, 3, 4, 4, 5, 5, 6, 6, 7, 7, 23]
[1, 1, 2, 2102, 2, 3, 3, 4, 4, 5, 5, 6, 6, 7, 7, 23]
[1, 1, 2, 2102, 2, 3, 3, 4, 4, 5, 5, 6, 6, 7, 7]
[1, 1, 2, 2, 3, 3, 4, 4, 5, 5, 6, 6, 7, 7]
[1, 1, 2, 2, 3, 3, 4, 5, 5, 6, 6, 7, 7]
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[50], line 13
     11 lst1.remove(4)       # remove the first occurance of the value 4
     12 print(lst1)
---> 13 lst1.remove(122)

ValueError: list.remove(x): x not in list
```

In [51]:
```
# Strings
course = "Artificial Intelligence"
print(course)
```

```
Artificial Intelligence
```

In [52]:
```
# String indexing is the same as lists (and slicing too)
print(course[0])
print(course[-3])
print(course[2:6])
```

```
A
n
tifi
```

String functions

There are many functions for strings, we will cover the basic ones

- `len()` - length of the string
- `.lower()` - convert all characters to lowercase
- `.upper()` - convert all characters to uppercase
- `.isalpha()` - True if the string has only alpha characters
- and many more to discover just like, `replace()`, `find()`, `partition()`, `format()` and many more !

In [53]:
```
# dictionaries: keys and values
# similar to hashmaps
dict1 = {'Egypt':'Cairo', "Saudi Arabia":'Ryiadh', 10:1, 16:[1,2]}
print(dict1)
```

```
{'Egypt': 'Cairo', 'Saudi Arabia': 'Ryiadh', 10: 1, 16: [1, 2]}
```

In [54]:
```
print(list(dict1.items()))
print(dict1.keys())
```

```
[('Egypt', 'Cairo'), ('Saudi Arabia', 'Ryiadh'), (10, 1), (16, [1, 2])]
dict_keys(['Egypt', 'Saudi Arabia', 10, 16])
```

In [55]:
```
dict1.keys()[0]
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[55], line 1
----> 1 dict1.keys()[0]

TypeError: 'dict_keys' object is not subscriptable
```

In [56]:
```
# if the key doesn't exist, it will be added with its assigned value
# but if you are looking for a specific key that it doesn't exist, it will raise an error
dict1[19] = 49
dict1
```

Out[56]:
```
{'Egypt': 'Cairo', 'Saudi Arabia': 'Ryiadh', 10: 1, 16: [1, 2], 19: 49}
```

In [57]:
```
dict1[252]
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[57], line 1
----> 1 dict1[252]

KeyError: 252
```

In [58]:
```
# Sets
st1 = {1,2,2,3,4,5,5,6,7,8}
st1
```

Out[58]:
```
{1, 2, 3, 4, 5, 6, 7, 8}
```

Comprehension

- a concise way to create sequences and collections using a single line of code. It simplifies loops and makes code more readable and efficient.

In [59]:
```
ls1 = [x**2 for x in range(1,9)]
ls1
```

Out[59]:
```
[1, 4, 9, 16, 25, 36, 49, 64]
```

In [60]:
```
# This is not Tuple comprehension !
tup2 = tuple(x for x in range(1,9))
tup2
```

Out[60]:
```
(1, 2, 3, 4, 5, 6, 7, 8)
```

In the prevous piece of code, it is **List comprehension** that is casted into tuple !

- There is no Tuple comprehension, since tuples are immutable !

In [61]:
```
dict2 = {x:y**2 for x,y in zip(range(1,9),range(1,9))}
dict2
```

Out[61]:
```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25, 6: 36, 7: 49, 8: 64}
```

(6) Selective structure and loops

Selective Structure

- A **selective structure** (also called a selection statement or decision-making structure) in Python is used to make choices in code execution based on conditions. It allows the program to selectively execute a block of code when a condition is met.

Types of selective structures:

1. `if` Statement → Executes a block of code if the condition is `True`.
2. `if-else` Statement → Chooses between two blocks of code.
3. `if-elif-else` Statement → Handles multiple conditions.
4. Nested `if` Statements → `if` inside another `if`.
5. Ternary Operator (`Conditional Expression`) → A short-hand `if-else`.

- Logical and Relational operators → `and`, `or`, `not`, `>`, `>=`, `<`, `<=`, `==` and `!=`

Loops

- a programming structure that repeatedly executes a block of code until a condition is met. Loops are used to avoid repetitive code and automate tasks efficiently.

1. for Loop → Iterates over a sequence (list, tuple, string, range, etc.).

2. while Loop → Repeats as long as a condition is True.

- Loop Control Statements → `break`, `continue`, `pass` (to control loop execution).

Indentation

- **Indentation**: the spaces or tabs used at the beginning of a line of code to define the structure of the program, especially in terms of code blocks.

In Python, indentation is not just for readability—it is syntactically important. It tells Python where a block of code begins and ends.

Why indentation is important ?

it **defines code blocks**, Indentation is used to define the grouping of statements. For example, a function's body or a loop's body is defined using indentation. And most importantly, it is syntax requirement, unlike other languages, Python requires indentation to separate blocks of code. Without it, Python will raise an **IndentationError**.

In [62]:
```
# check if x greater than 5 or not
if x < 5 or y > 2:
    print('yes')
else:
    print('no')
```

```
yes
```

In [63]:
```
# check grade
grade = 76
if grade >= 90:
    print("A+")
elif grade < 90 and grade >= 85:
    print("A")
elif grade < 85 and grade >= 80:
    print("B+")
elif grade < 80 and grade >= 75:
    print("B")
```

```
B
```

In [64]:
```
# Check this out
# note that "print(grade) is outside the scope of the if
if grade < 85:

print(grade)
```

```
  Cell In[64], line 5
    print(grade)
    ^
IndentationError: expected an indented block after 'if' statement on line 3
```

In [65]:
```
if grade < 85:
    print(grade)

# or to overcome the previos problem and make the "if" scope empty, use pass

if grade < 85:
    pass
```

```
76
```

In [66]:
```
# using short-hand if-else
dum_list = [x if x > 5 else 0 for x in range(20)]
dum_list
```

Out[66]:
```
[0, 0, 0, 0, 0, 0, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19]
```

In [67]:
```
#Loops
x = 20
while x > 5:
    print(x)
    x -= 1
else:
    print("Hello")
```

```
20
19
18
17
16
15
14
13
12
11
10
9
8
7
6
Hello
```

In [68]:
```
for x in range(len(lst1)):
```

```
    print(x)
```

```
0
1
2
3
4
5
6
7
8
9
10
11
12
```

- As noticed, since **range()** generates iterable, any other iterable [`zip()` , `map()`] can be used with loops

(7) Functions

- A **function** is a reusable block of code that performs a specific task. Functions help organize code, avoid repetition, and make programs more readable and efficient.

just use the keyword `def` to define any function

In [69]:
```
def myfunc(x,y,z):
    return x+y+z
print(myfunc(1,2,3))
```

```
6
```

In [70]:
```
def list_modifying(lst, x):
    x = 11
    lst[3] = x
    return lst
x = 25
my_list1 = [10,5,3,21]
print(my_list1, x)
print(list_modifying(my_list1, x))
print(my_list1, x)
```

```
[10, 5, 3, 21] 25
[10, 5, 3, 11]
[10, 5, 3, 11] 25
```

Note: Sequences and collections are passed **by reference** to functions, while variables are passed **by value**

- to overcome this, you may use `.copy()` method of lists

In [71]:
```
# This is not Function Overloading, IT IS NOT SUPPORTED IN PYTHON ! Only the recent function definition will be used, the old one is erased
def list_modifying(lst, x):
    x = 11
    lst[3] = x
    return lst

def list_modifying(lst):
    dum_list_fun = lst.copy()
    dum_list_fun[2] = 2134
    return dum_list_fun

my_list2 = [2,3,4,5]
print(list_modifying(my_list2))
print(my_list2)
print(list_modifying(my_list2,x)) # Raise an error !
```

```
[2, 3, 2134, 5]
[2, 3, 4, 5]
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[71], line 15
     13 print(list_modifying(my_list2))
     14 print(my_list2)
---> 15 print(list_modifying(my_list2,x))

TypeError: list_modifying() takes 1 positional argument but 2 were given
```

(7) Classes

- A **class** is a blueprint for creating objects. It defines attributes (data) and methods (functions) that describe the behavior of an object.
- use `class` keyword to define a class

Key concepts:

1. Class → A blueprint for objects.
2. Object → An instance of a class.
3. Attributes (Variables) → Store object data (`self.brand`).

4. Methods (Functions in Class) → Define object behavior (`display_info()`).
5. Constructor (`__init__`) → Initializes object attributes.

Note: you may need to use `self` to let the interpreter understands that the **attributes** and **methods** will belong to each created object. without it it will be considered as **static** or **Class variable** which is shared with all instances !

Encapsulation

- Use private attributes (`__attribute`) to hide data.

In [72]:
```python
class Dog:
    species = "Mammal"  # Class Variable

    def __init__(self, name, age):
        self.name = name  # Instance Variable
        self.__age = age  # Private attribute


dog1 = Dog("Buddy", 2)
dog2 = Dog("Max", 1)

print(dog1.name)  # Output: Buddy
print(dog2.name)  # Output: Max
print(dog1.species, dog2.species) # Output: Mammal Mammal
Dog.species = "German"
print(dog1.species, dog2.species)
print(dog1.age)
```

```
Buddy
Max
Mammal Mammal
German German
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[72], line 17
     15 Dog.species = "German"
     16 print(dog1.species, dog2.species)
---> 17 print(dog1.age)

AttributeError: 'Dog' object has no attribute 'age'
```

In [73]:
```python
# This is very Tricky !
dog1.species = "Mammal"
print(dog1.species, dog2.species)
```

```
Mammal German
```

In [74]:
```python
print(dog1.__dict__)
print(dog2.__dict__)
```

```
{'name': 'Buddy', '_Dog__age': 2, 'species': 'Mammal'}
{'name': 'Max', '_Dog__age': 1}
```

1. `.__dict__` is a built-in method for each object that displays all attributes of an object
2. in the previous piece of code `dog1` has a new **DIFFERENT** attribute called `species` but not the `class variable` which is assigned `Mammal` , while `dog2` hasn't attribute `species` so it used `class variable` which is `German` !
3. the `age` attribute isn't actually private attribute, it can be accessed using the shown name in the previus output !

In [75]:
```python
dog1._Dog__age
```

Out[75]: 2

In [76]:
```python
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance  # Private Variable

    def get_balance(self):
        return self.__balance  # Controlled Access

account = BankAccount(1000)
print(account.get_balance())  # Output: 1000
```

```
1000
```

In [77]:
```python
# Inheritance
class Animal:
    def __init__(self, species):
        self.species = species
        print(f"Animal of species {self.species} is created.")

class Dog(Animal):
    def __init__(self, species, breed):
        # Call the parent class constructor
        super().__init__(species)            # super() keyword in order to refer to parent class constructor
        self.breed = breed
        print(f"Dog of breed {self.breed} is created.")

# Creating an object of Dog
```

```python
dog = Dog("Canine", "Labrador")

# Output:
# Animal of species Canine is created.
# Dog of breed Labrador is created.
```

```
Animal of species Canine is created.
Dog of breed Labrador is created.
```

In [78]:
```python
# Multiple-Inheritance
class Parent1:
    def __init__(self):
        self.var = "Benha University"

class Parent2:
    def __init__(self):
        self.var = "FCAI"

class Child(Parent1, Parent2): #The order of inheritance is CRITICAL to avoid Diamond Problem (Method Resolution Order -MRO-)
    def __init__(self):
        super().__init__()
        print(self.var) # Whose (var) will be printed ????????

x = Child()
```

```
Benha University
```

In [79]:
```python
# Another example but with reversed MRO
class Child(Parent2, Parent1): #Reversed order
    def __init__(self):
        super().__init__()
        print(self.var) # Whose (var) will be printed ????????

x = Child()
```

```
FCAI
```

- If incosistency found during Multiple-Inheritance, "MRO" allow for **((First Parent Only))** to pass through its copy
- It is a method **"MRO"** instead of `Virtual Inheritance` in C++

In [80]:
```python
#Polymorphism
class Vehicle:
    def __init__(self, brand, model):
        self.model = brand
        self.brand = model

    def move(self): # No need to define a pure virtual function (just like C++) -Dynamicity Feature-
        pass

    def info(self):
        print(self.model, self.brand)

class Car(Vehicle):
    def __init__(self, brand, model):
        super().__init__(brand,model)

    def move(self): #Function overriding
        print("Move !")

class Boat(Vehicle):
    def __init__(self, brand, model):
        super().__init__(brand,model)

    def move(self): #Function overriding
        print("Sail !")

class Plane(Vehicle):
    def __init__(self, brand, model):
        super().__init__(brand,model)

    def move(self): #Function overriding
        print("Fly !")

# No need to change info() method since it is common
```

In [81]:
```python
car1 = Car("Ford", "Mustang")
boat1 = Boat("Ibiza", "Touring 20")
plane1 = Plane("Boeing", "747")
for x in (car1, boat1, plane1): # no need to define (x) as reference to parent class -Dynamicity Feature-
    x.info()
    x.move()
```

```
Ford Mustang
Move !
Ibiza Touring 20
Sail !
Boeing 747
Fly !
```

(8) Exercises (Activity)

Next Lab: Searching Problems (1)

Thank you